PROGRAMACIÓN 5 · SEMESTRE 5

# BioMonitor $O_2T$

Monitoreo de signos vitales con ESP32, separación por usuario y auditoría


| | |
|---|---|
| **INSTITUCIÓN** | [Completar] |
| **MATERIA** | Programación 5 |
| **SEMESTRE** | 5.º |
| **INTEGRANTES** | [Completar nombres] |
| **DOCENTE** | [Completar] |
| **FECHA** | 4 de noviembre de 2026 |


Informe final · Documento académico

# Resumen ejecutivo

BioMonitor $O_2T$ es una aplicación cliente-servidor para monitorear oxigenación ($SpO_2$), pulso y temperatura con dispositivos ESP32. El sistema permite que dos usuarios y dos dispositivos transmitan al mismo tiempo, mantiene los datos separados por propietario, aplica permisos en el servidor y muestra el origen de cada lectura, la última recepción, el estado de conexión, el historial y una gráfica con banda de referencia.

La versión final incorpora autenticación con cookie JWT, contraseñas Argon2id, credenciales de dispositivo con HMAC-SHA256, emparejamiento por código temporal, auditoría de accesos y rechazos, y un firmware que lee un MAX30102 real y un DS18B20 opcional. La simulación quedó como una opción explícita y etiquetada.

La suite automática cerró con **62 pruebas aprobadas**. La aplicación se verificó en navegador con datos demo y con el simulador de dos dispositivos. La validación con hardware físico sigue pendiente porque no se dispone de los módulos y el cableado definitivo al momento de cerrar el informe.

**Palabras clave:** ESP32, MAX30102, $SpO_2$, FastAPI, RBAC, HMAC, telemetría.

# 1. Introducción

El monitoreo de signos vitales en casa requiere algo más que mostrar números: hace falta saber si el dato es actual, de qué sensor proviene, si el equipo sigue conectado y quién puede verlo. Una lectura sin fecha, sin origen o sin control de acceso puede llevar a decisiones equivocadas.

Este proyecto construye un prototipo académico que responde a esas preguntas. El paciente consulta su estado; un cuidador o especialista revisa únicamente a las personas asignadas; el administrador gestiona sensores y vínculos; el auditor revisa la actividad. El servidor es la autoridad de permisos y de asociación entre dispositivos y cuentas.

## 1.1 Objetivo general

Implementar una aplicación cliente-servidor que reciba datos de dos ESP32, los separe por usuario, los muestre con su origen y estado de conexión, y registre la actividad relevante para auditoría.

## 1.2 Objetivos específicos

1. Autenticar usuarios y aplicar RBAC en el servidor.
2. Asociar cada ESP32 a una cuenta mediante credenciales y emparejamiento.
3. Recibir lecturas reales o simuladas sin confundir su origen.
4. Mostrar última lectura, fecha/hora, última recepción y estado de conexión.
5. Presentar historial y gráficas de $SpO_2$, pulso y temperatura.
6. Auditar accesos, rechazos y cambios de asociación sin registrar secretos.
7. Documentar frecuencias, red, instalación y demostración simultánea.

# 2. Requisitos y alcance

| Requisito | Decisión de diseño |
|---|---|
| Login/logout | JWT en cookie HttpOnly, expiración y logout del servidor. |
| RBAC | Dependencias `require_roles` en todas las rutas sensibles. |
| Sin cruces entre usuarios | Propietario tomado de `devices.user_id`; historial filtrado por dueño actual. |
| Dos usuarios y dos ESP32 | Un dispositivo por cuenta; simulador con dos hilos y claves distintas. |
| Origen de cada registro | Campo `source` obligatorio: `esp32`, `sensor_die`, `simulator`, `demo`. |
| Credencial por dispositivo | `api_key` aleatoria, guardada como HMAC-SHA256. |
| Última recepción y estado | `last_seen` + última lectura con ventanas de 15 s y 90 s. |
| Historial y gráfica | SVG con eje temporal, unidad, banda de referencia y resumen accesible. |
| Auditoría | Eventos `LOGIN`, `LOGOUT`, `ACCESS_DENIED`, `DEVICE_*`. |
| Secretos fuera del código | `.env`, `config.py` y `device_config.json` ignorados por Git. |

**Fuera de alcance:** diagnóstico médico, recetas, historia clínica completa, notificaciones push, aplicación móvil nativa y validación clínica.

# 3. Arquitectura

```
ESP32 / simulador
   │  HTTPS/HTTP local + X-Device-Api-Key
   ▼
FastAPI ──► SQLAlchemy ──► SQLite
   │
   ├── Autenticación JWT (cookie HttpOnly)
   ├── RBAC por rol
   ├── Servicios de dispositivos, cuidado y mediciones
   ├── Auditoría
   └── Frontend estático (HTML/CSS/JS)
```

### 3.1 Componentes

- **Frontend:** HTML, CSS y JavaScript sin framework. Renderiza el panel por rol, consume la API con `fetch` y actualiza cada 10 s.
- **Backend:** FastAPI + SQLAlchemy. Valida contratos con Pydantic, aplica permisos y centraliza las reglas de negocio.
- **Base de datos:** SQLite para el prototipo. Tablas principales: `users`, `devices`, `measurements`, `care_assignments`, `audit_logs`.
- **Firmware:** MicroPython. MAX30102 por I2C, DS18B20 opcional por OneWire, aprovisionamiento, ingestión y heartbeat.
- **Simulador:** dos hilos, dos claves, dos propietarios, `source="simulator"`.

### 3.2 Modelo de datos relevante

| Tabla | Campos principales |
|---|---|
| `users` | `id`, `full_name`, `email`, `password_hash`, `role`, `is_active`. |
| `devices` | `id`, `device_id`, `api_key` (HMAC), `user_id`, `is_active`, `last_seen`, `pairing_code_hash`, `pairing_expires_at`. |
| `measurements` | `id`, `user_id`, `device_id`, `spo2`, `heart_rate_bpm`, `temperature_c`, `source`, `measured_at`. |
| `care_assignments` | `patient_id`, `member_id`, `relationship`, `is_active`. |
| `audit_logs` | `action`, `status`, `detail`, `user_id`, `email_attempted`, `ip_address`, `created_at`. |

La medición guarda `user_id` en el momento de la ingesta. Si el dispositivo se reasigna, el historial se consulta con el propietario actual para no exponer datos de la cuenta anterior.

## 4. Seguridad

### 4.1 Autenticación y sesión

- Contraseñas con **Argon2id** mediante `pwdlib`.
- JWT firmado con `SECRET_KEY`, almacenado en cookie **HttpOnly**.
- Expiración configurable y logout que elimina la cookie.
- Los errores de login no revelan si el correo existe.

### 4.2 Autorización

- RBAC evaluado en el servidor con `require_roles`.
- El paciente solo accede a sus dispositivos.
- Cuidador y especialista solo acceden a pacientes con vínculo activo.
- Administrador y auditor tienen vistas específicas; el auditor es de solo

lectura.

- El frontend oculta acciones, pero no es la barrera de seguridad.

### 4.3 Credenciales de dispositivo

- La `api_key` se genera con `secrets.token_urlsafe`.
- Se guarda como **HMAC-SHA256** usando `SECRET_KEY` como pepper.
- Solo se muestra en claro al registrar o rotar.
- El código de emparejamiento también se guarda como HMAC, vence a los 10

minutos y es de un solo uso.

- La ingesta exige dispositivo vinculado y cuenta activa; el propietario se lee

de la base de datos, nunca del payload.

### 4.4 Auditoría

Se registran, entre otros:

- `LOGIN`, `LOGOUT`, `ACCESS_DENIED`.
- `DEVICE_PROVISIONED`, `DEVICE_REGISTERED`, `DEVICE_PAIRED`,

`DEVICE_PAIR_FAILED`, `DEVICE_KEY_ROTATED`, `DEVICE_UNPAIRED`.

- `DEVICE_AUTH_FAILED` y `DEVICE_DATA_REJECTED`.

No se registran contraseñas, tokens ni valores del payload. Los rechazos de ingesta anotan el dispositivo y el motivo, no la credencial.

---

## 5. Adquisición y visualización

### 5.1 Firmware

- **MAX30102:** configuración $SpO_2$ a 100 Hz, FIFO de 32 muestras, ventana de 4 s.
- **$SpO_2$:** estimación por relación AC/DC (`110 − 25 × R`), aproximación de banco.
- **Pulso:** detección de picos del PPG infrarrojo con periodo refractario.
- **Temperatura:** DS18B20 opcional; si falta, se envía la temperatura interna

del MAX30102 con `source="sensor_die"`.

- **Sin lectura válida:** no se envía nada. No se inventan valores.
- **Simulación:** solo con `SIMULATION_MODE=True`, marcada como `simulator`.

### 5.2 Frecuencias

| Etapa | Valor |
|---|---|
| Muestreo PPG | 100 Hz durante 4 s |
| Envío al servidor | cada 5 s |
| Heartbeat | cada 60 s |
| Refresco del panel | cada 10 s |
| Dato “en vivo” | última lectura ≤ 15 s |
| Enlace activo | último contacto ≤ 90 s |

La latencia observada entre la ingesta y la aparición en el panel es de 0 a 10 s por el refresco automático, más el tiempo de red. El botón **Actualizar** reduce la espera a la latencia de red.

### 5.3 Interfaz

- Tarjetas por sensor con última recepción, última lectura, valores y origen.
- Estados: recibiendo, en línea sin lectura nueva, desconectado, desactivado y

sin lecturas.

- Historial de 10, 30, 60 o 100 lecturas.

- Tres gráficas SVG con banda de referencia y resumen de mínimo, máximo y último valor.
- Tabla con fecha/hora, sensor, valores, origen y estado.
- Vistas de cuidador/especialista con el mismo historial para pacientes asignados.

# 6. Resultados

## 6.1 Pruebas automáticas

```
python -m pytest -q
62 passed
```

Cobertura funcional:

- Autenticación, logout y sesión.
- RBAC y accesos denegados.
- Vínculos de cuidado y visibilidad por rol.
- Registro, aprovisionamiento, emparejamiento y rotación de claves.
- Límites de capacidad.
- Ingesta no vinculada y credenciales inválidas.
- Aislamiento de historial entre propietarios.
- Auditoría de eventos.

## 6.2 Verificación en navegador

Se comprobó:

- Login y restauración de sesión.
- Panel de paciente con gráficas e historial.
- Panel de cuidador y detalle de paciente con gráficas.
- Panel de administrador con estado de cada sensor.
- Panel de auditoría y filtros.
- Refresco automático cada 10 s.
- Cambio de rango de historial.
- Simulador de dos dispositivos con `source="simulator"`.

## 6.3 Resultados esperados vs obtenidos

| Requisito | Esperado | Obtenido | Corrección |
|---|---|---|---|
| Login/logout | Sesión segura | Cumplido | Cookie HttpOnly. |
| RBAC | Permisos en servidor | Cumplido | `require_roles`. |

| Requisito | Esperado | Obtenido | Corrección |
|---|---|---|---|
| Dos usuarios/dispositivos | Datos separados | Cumplido con simulador; hardware pendiente | Dueño desde BD. |
| Origen del dato | Visible y fiable | Cumplido | `source` + etiquetas. |
| Credenciales | Sin texto plano | Cumplido | HMAC-SHA256. |
| Ingesta sin vincular | Rechazada | Cumplido | 409 + auditoría. |
| Historial tras reasignación | Sin fuga | Cumplido | Filtro por dueño actual. |
| Historial/gráfica | Tendencia clara | Cumplido | SVG con referencia. |
| Desconexión | Estado visible | Cumplido | 15/90 s. |
| Firmware real | Sin simulación por defecto | Implementado, sin validar | MAX30102/DS18B20. |

### 6.4 Correcciones realizadas durante el cierre

1. Las claves de dispositivo se guardaban en texto plano; ahora usan HMAC.
2. La semilla marcaba datos demo como `esp32`; ahora usa `demo`.
3. La ingesta aceptaba dispositivos sin dueño; ahora responde `409` y audita.
4. El historial podía mostrar lecturas del dueño anterior; ahora filtra por propietario actual.
5. El firmware generaba valores aleatorios por defecto; ahora solo lee hardware o simula si se activa explícitamente.
6. El panel no tenía gráficas ni selector de historial; se agregaron.
7. El refresco automático estaba apagado y decía “cada minuto”; ahora está activo cada 10 s.
8. Los textos “Datos de ejemplo” se reemplazaron por origen dinámico.

## 7. Matriz de cumplimiento

| Requisito | Estado | Evidencia |
|---|---|---|
| Login/logout | Cumplido | `/api/auth/*`, pruebas. |
| RBAC en servidor | Cumplido | Pruebas de 403. |
| Sin lecturas cruzadas | Cumplido | `test_final_integration.py`. |
| Dos usuarios y dos ESP32 | Cumplido con simulador; hardware pendiente | `simulator/device_simulator.py`. |
| Origen de cada registro | Cumplido | UI y tabla. |
| Credencial por dispositivo | Cumplido | HMAC y rotación. |
| Asociación validada en servidor | Cumplido | `user_id` desde BD. |
| Última lectura y recepción | Cumplido | Tarjetas y tablas. |
| Estado de actualización/desconexión | Cumplido | `deviceStatusInfo`. |
| Historial y gráfica | Cumplido | SVG $SpO_2$/pulso/temperatura. |

| Requisito | Estado | Evidencia |
|---|---|---|
| Frecuencias y retardo | Cumplido | Tabla de la sección 5.2. |
| Contraseñas hasheadas | Cumplido | Argon2id. |
| Secretos fuera del código | Cumplido | `.env` y `.gitignore`. |
| Auditoría | Cumplido | `audit_logs`. |
| Documentación | Cumplido | README y guía de dispositivos. |
| PDF, PPTX y video | Parcial | PDF/PPTX generados; video pendiente. |

# 8. Discusión

La decisión más importante fue **no confiar en el payload** para determinar al propietario. Si el ESP32 pudiera enviar `user_id`, un dispositivo comprometido podría escribir en el historial de otra persona. Al leer el dueño de `devices.user_id` y exigir un dispositivo vinculado, la separación queda del lado del servidor.

La segunda decisión fue **no inventar lecturas**. Un sensor óptico puede fallar por movimiento, mala colocación o señal débil. Enviar un valor aleatorio en ese caso sería peor que no enviar nada, porque el usuario no podría distinguir un dato real de uno fabricado. Por eso el firmware devuelve `None` y el panel muestra el último dato con su fecha y estado.

La tercera fue **hacer visible el origen**. Los datos demo, el simulador y el ESP32 real conviven en el mismo historial durante la demostración. La etiqueta `source` permite explicar qué se está viendo sin confundir la simulación con una medición física.

# 9. Limitaciones

1. No se validó el MAX30102 ni el DS18B20 con hardware definitivo.
2. La fórmula $SpO_2$ es una aproximación y no un algoritmo certificado.
3. SQLite es suficiente para el prototipo, pero no para concurrencia alta.
4. No hay notificaciones push ni aplicación móvil nativa.
5. La creación de cuentas desde la interfaz queda fuera de esta iteración.
6. El video de respaldo debe grabarse antes de la entrega.

# 10. Conclusiones

El proyecto cumple los requisitos funcionales y de seguridad centrales: dos usuarios pueden operar dos dispositivos al mismo tiempo, el servidor separa los datos, cada lectura conserva su origen y el panel muestra actualización, desconexión, historial y tendencia. La suite de 62 pruebas y la verificación en navegador respaldan el comportamiento.

El siguiente paso es la validación física: conectar los módulos, comparar la $SpO_2$ con un oxímetro de referencia, ajustar la calibración y grabar el video de respaldo con dos sesiones y dos dispositivos.

# Referencias


- FastAPI. *Documentation*. https://fastapi.tiangolo.com/
- SQLAlchemy. *Documentation*. https://docs.sqlalchemy.org/
- MicroPython. *ESP32 documentation*. https://docs.micropython.org/en/latest/esp32/
- Analog Devices. *MAX30102 datasheet*.
- Maxim Integrated. *DS18B20 datasheet*.
- OWASP. *Application Security Verification Standard*.


# Anexo A — Comandos de ejecución

```
python -m pip install -r backend/requirements.txt
cd backend
python -m uvicorn app.main:app --reload --host 0.0.0.0 --port 8100
```

```
python -m pytest -q
node --check frontend/assets/app.js
python simulator/prepare_demo.py
python simulator/device_simulator.py
```

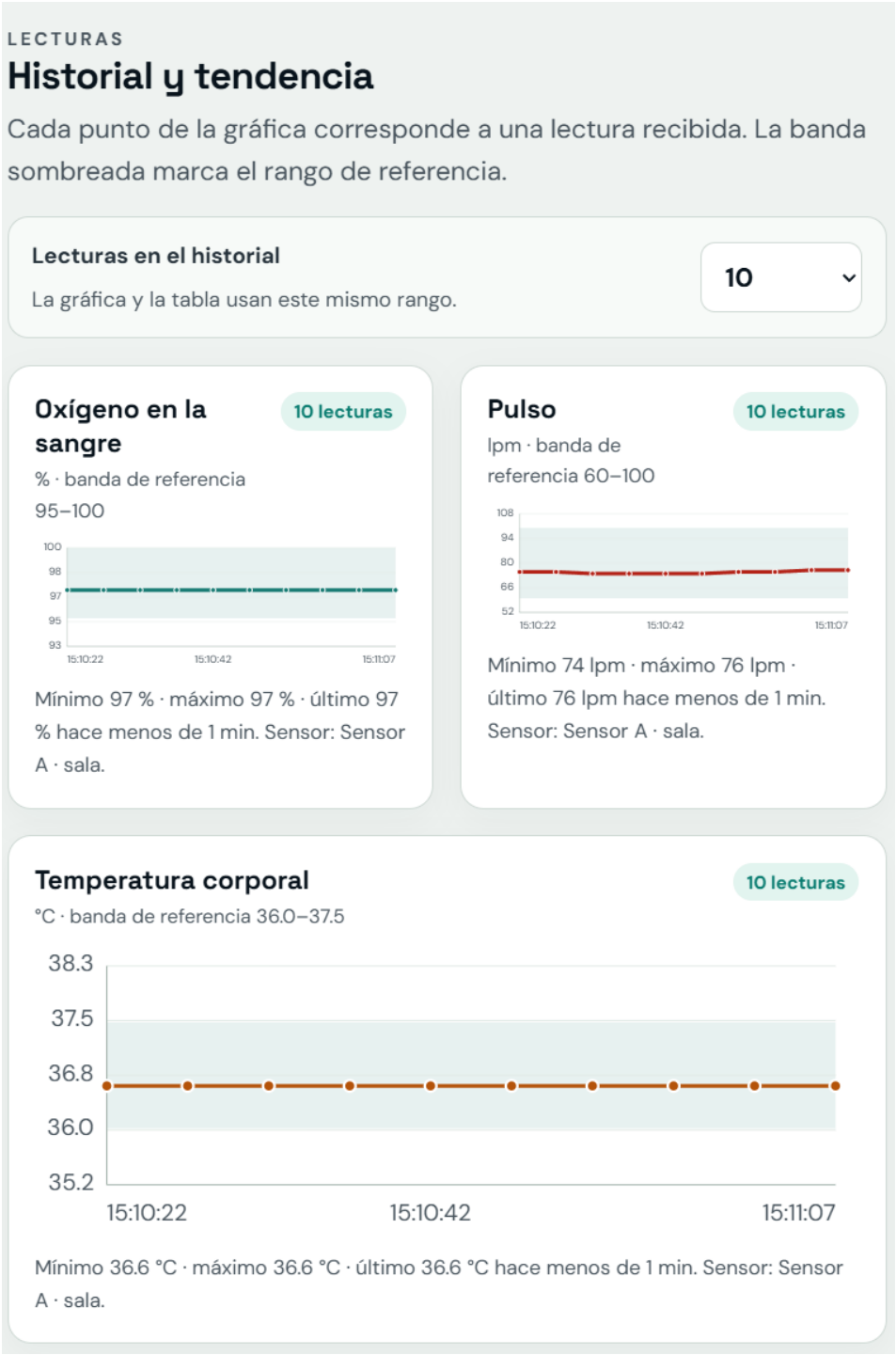
Las capturas de la aplicación se incluyen en el anexo B del PDF generado.

# Anexo B — Capturas de la aplicación

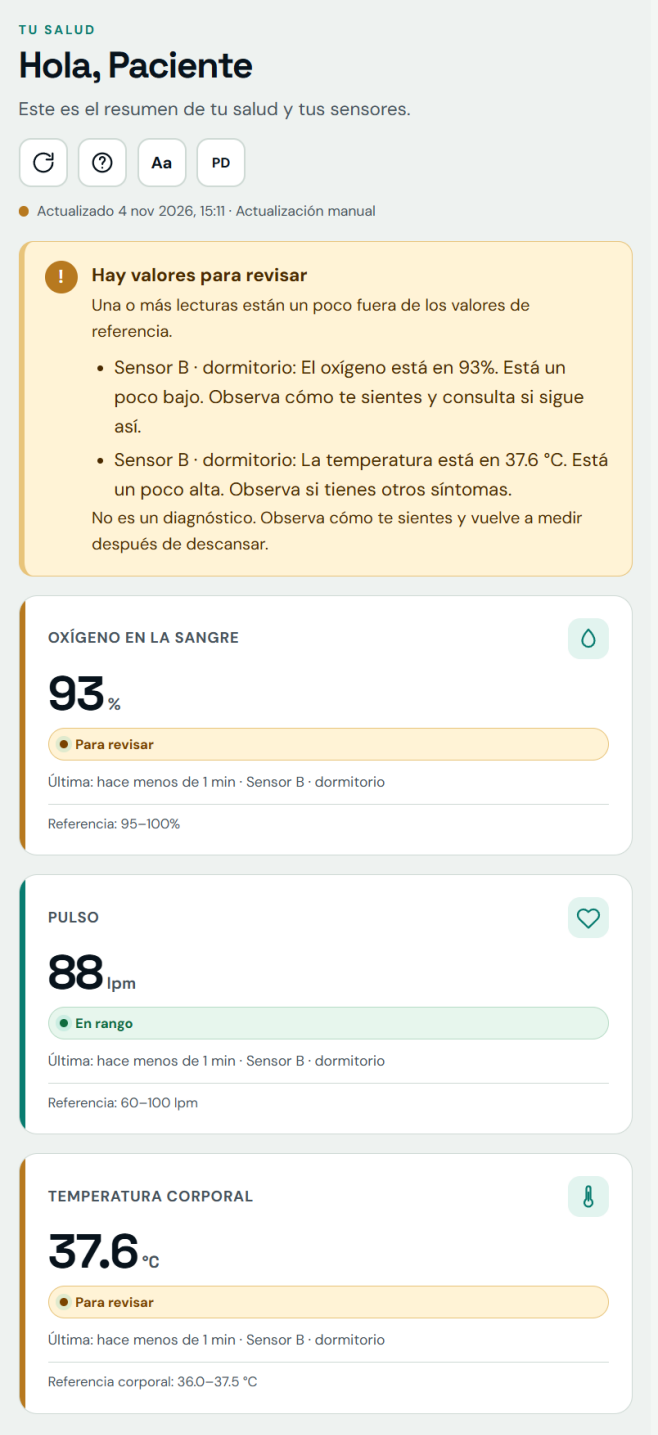
Capturas tomadas durante la verificación final en navegador. Los datos mostrados son de demostración o del simulador y se identifican como tales en la interfaz.




*Panel del paciente: valores actuales, estado y última recepción.*




*Historial y gráficas del paciente con 10 lecturas del simulador.*




*Segunda sesión de paciente: valores distintos y su propio sensor.*




*Gráficas de la segunda sesión, separadas de la primera.*




*Detalle de paciente en el panel de cuidador.*




*Historial y gráficas dentro del detalle del cuidador.*

SENSORES

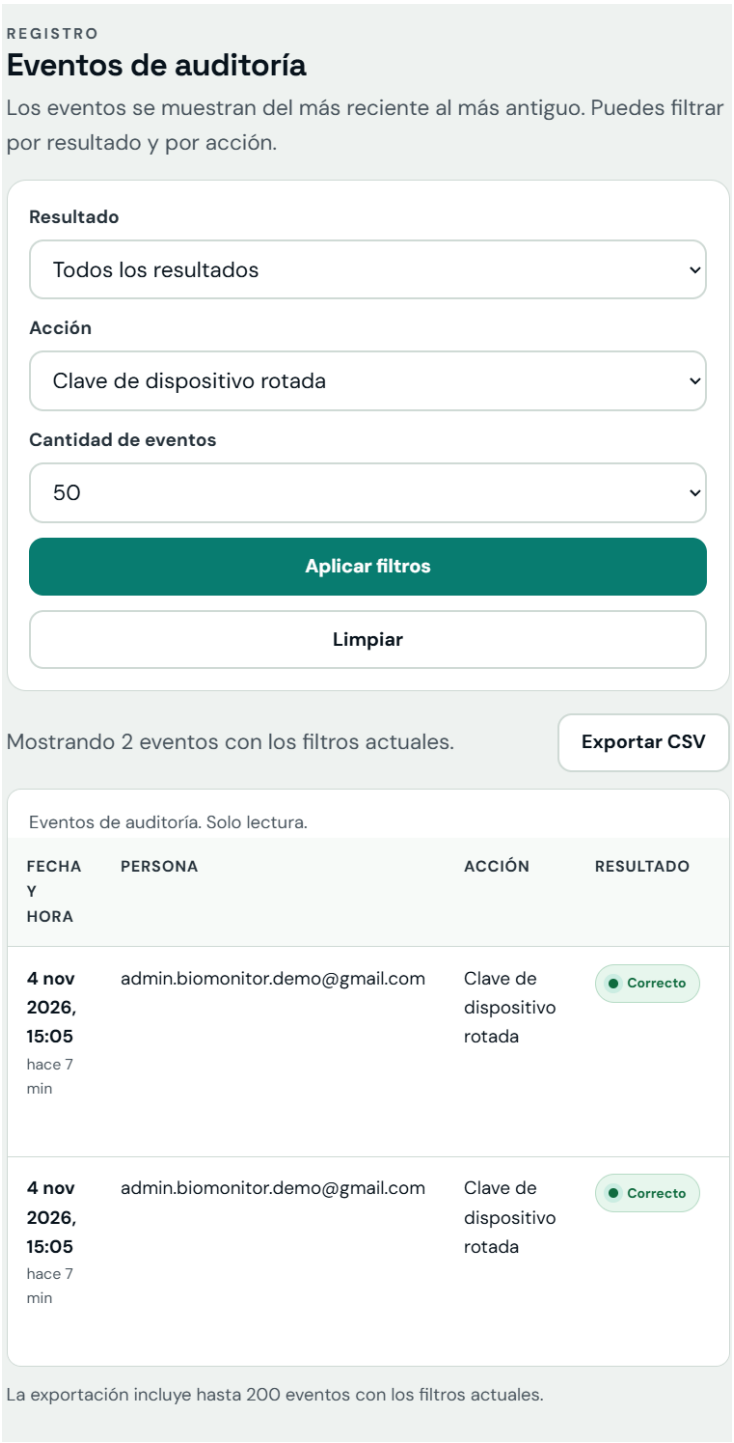
## Dispositivos registrados

Edita el nombre, la ubicación o el estado de un sensor. Desactivarlo no borra sus lecturas anteriores.

Sensores registrados. Puedes editarlos o cambiar su estado.

| SENSOR | PROPIETARIO | UBICACIÓN | ESTADO | ÚLTIMO CONTACTO |
|---|---|---|---|---|
| **Sensor A · sala** ESP32-O2T-01 | Paciente Demo | Sala de estar | En línea | hace menos de 1 min |
| **Sensor B · dormitorio** ESP32-O2T-02 | Paciente Demo Dos | Dormitorio | En línea | hace menos de 1 min |
| **Sensor C · reserva** ESP32-O2T-03 | Paciente Demo Tres | Sin ubicación | Sin contacto | Sin contacto |

*Panel de administración con el estado de contacto de los sensores.*




*Auditoría filtrada por eventos de dispositivo.*